

JAVA · DOKUMENTACJA AWARYJNA

Od OOP do JavaFX

Kompletne podsumowanie materiału — z rozszerzonymi sekcjami dla Map, Iteratorów i switch. Do szybkiego sprawdzenia, nie do czytania od A do Z.

wersja roboczo-egzaminacyjna

Spis treści

1. OOP — podstawy	str. 3
> Klasy, obiekty, konstruktory	str. 3
> Modyfikatory dostępu	str. 3
> Dziedziczenie i polimorfizm	str. 3
> Klasy abstrakcyjne vs interfejsy	str. 4
> Enum	str. 4
> Record	str. 5
> Generyki	str. 5
2. Kolekcje — dogłębnie	str. 6
> List, Set, Map — przegląd	str. 6
> MAP — pełny rozdział	str. 6
> ITERATOR — pełny rozdział	str. 8
> Sortowanie kolekcji	str. 9
3. SWITCH — pełny rozdział	str. 10
> Switch klasyczny	str. 10
> Switch nowoczesny (arrow)	str. 10
> Switch jako wyrażenie	str. 11
4. Wyjątki	str. 13
> Checked vs unchecked	str. 13
> try-catch-finally	str. 13
> try-with-resources	str. 13
> Własne wyjątki	str. 13
5. JUnit	str. 14
> Struktura testu	str. 14
> Asercje	str. 14
6. Programowanie funkcyjne	str. 15
> Lambdy	str. 15
> Interfejsy funkcyjne	str. 15
> Wzorzec callback	str. 15
> Stream API	str. 15
7. Wątki i synchronizacja	str. 16
> Thread, Runnable, start/run	str. 16
> ExecutorService	str. 16
> Synchronizacja	str. 16
8. Sockets — serwer i klient	str. 17
> ServerSocket, accept()	str. 17
> Pełny wzorzec Server+ClientHandler	str. 17
> Klient TCP	str. 18
> Pliki binarne przez socket	str. 18
9. JavaFX	str. 19
> Stage, Scene, layouty	str. 19
> Kontrolki i listenery	str. 19
> Canvas	str. 20
> Klawiatura i mysz	str. 20
Checklist ratunkowy	str. 21

1 OOP — podstawy

Klasy, obiekty, konstruktory

Klasa to przepis, **obiekt** to konkretna rzecz zrobiona według tego przepisu. Każde `new` tworzy nowy, niezależny obiekt z własną kopią pól.

```
public class Player {
    private String login;          // pole instancyjne – każdy obiekt ma SWOJĄ kopię
    private static int totalPlayers = 0; // pole statyczne – WSPÓLNE dla wszystkich obiektów

    public Player(String login) {
        this.login = login;
        totalPlayers++;
    }

    // konstruktor uproszczony – woła inny przez this(...)
    public Player() {
        this("anonim");
    }
}
```

Czemu `main()` jest `static`: w momencie startu programu nie istnieje jeszcze żaden obiekt, więc metoda startowa nie może "należać" do obiektu.

Pułapka: `this(...)` musi być PIERWSZĄ instrukcją w konstruktorze.

Modyfikatory dostępu

Modyfikator	Widoczność
<code>private</code>	tylko w tej samej klasie
(brak)	w tym samym pakiecie (package-private)
<code>protected</code>	pakiet + klasy dziedziczące (nawet w innym pakiecie)
<code>public</code>	wszędzie

Zasada minimalnej widoczności: wybieraj zawsze najbardziej restrykcyjny poziom, który jeszcze pozwala programowi działać.

Dziedziczenie i polimorfizm

```
class Player {
    protected String login;
    public Player(String login) { this.login = login; }
    public void greet() { System.out.println("Hej, " + login); }
}

class ClientHandler extends Player {
    public ClientHandler(String login) {
        super(login); // MUSI być pierwsza instrukcja
    }
    @Override
    public void greet() {
        super.greet(); // wykonaj wersję rodzica
        System.out.println("+ dodatkowe zachowanie");
    }
}
```

Polimorfizm — zmienna typu rodzica może wskazywać na obiekt podklasy, a JVM sam wybiera właściwą (nadpisaną) wersję metody w czasie działania:

```
List<Player> all = new ArrayList<>();
all.add(new Player("ania"));
all.add(new ClientHandler("bartek"));
for (Player p : all) { p.greet(); } // dla ania -> Player.greet(), dla bartek -> ClientHandler.greet()
```

Klasy abstrakcyjne vs interfejsy

Klasa abstrakcyjna — gdy chcesz dać podklasom część WSPÓLNEJ implementacji + wymusić resztę. Nie można zrobić `new`.

```
abstract class Shape {
    abstract double area();           // BEZ ciała – podklasa MUSI to zaimplementować
    public void describe() { System.out.println("Pow: " + area()); } // Z ciałem – wspólne
}
```

Interfejs — wymusza "kontrakt" (co klasa MUSI umieć). Klasa może implementować WIELE interfejsów (ale dziedziczyć tylko po JEDNEJ klasie).

```
interface Drawable {
    void draw();                     // kontrakt
    default void highlight() { System.out.println("~"); } // metoda domyślna, opcjonalna do nadpisania
}
```

Reguła praktyczna: różne rzeczy robiące to samo, bez wspólnego kodu → interfejs. Rodzina podobnych obiektów z dużą częścią wspólnego kodu → klasa abstrakcyjna.

Enum

Skończony, znany z góry zbiór wartości. Może mieć własne pola, konstruktory, metody.

```
enum Gesture {
    ROCK, PAPER, SCISSORS;

    static Gesture fromString(String s) {
        return switch (s) {
            case "r" -> ROCK;
            case "p" -> PAPER;
            case "s" -> SCISSORS;
            default -> throw new IllegalArgumentException(s);
        };
    }

    int compareWith(Gesture other) {
        if (this == other) return 0;
        return ((this.ordinal() + 1) % 3 == other.ordinal()) ? 1 : -1;
    }
}
```

`ordinal()` = pozycja w deklaracji (ROCK=0, PAPER=1, SCISSORS=2).

Record

Niemutowalny nośnik danych — Java generuje konstruktor, gettersy (bez "get"), `equals/hashCode/toString` automatycznie.

```
record Result(Player winner, Player loser) {}

Result r = new Result(playerA, playerB);
Player w = r.winner(); // getter BEZ "get"
```

Rekord dobry, gdy dane raz ustawione się nie zmieniają. Można go zagnieździć wewnątrz klasy (`class Duel { record Result(...) {} }`).

Generyki

Jedna klasa działająca z różnymi typami, z bezpieczeństwem sprawdzanym na etapie KOMPILACJI (nie runtime).

```
class Box<T> {
    private T content;
    public void set(T content) { this.content = content; }
    public T get() { return content; }
}
```

```
Box<String> nameBox = new Box<>();    // T = String DLA TEGO obiektu
nameBox.set("ania");
// nameBox.set(5);  <- BŁĄD KOMPILACJI, T jest ustalone na String
```

Metoda generyczna: `static <T> T firstElement(List<T> list) { return list.get(0); }` — działa dla `List<String>`, `List<Player>`, itd.

2 Kolekcje — dogłębnie

List, Set, Map — przegląd

Typ	Cecha	Kiedy używać
List	uporządkowana, duplikaty OK, dostęp po indeksie	kolejność ważna, duplikaty możliwe (lista wiadomości, kolejka klientów)
Set	brak duplikatów, brak indeksów	gwarancja unikalności (zbiór zalogowanych loginów)
Map	klucz→wartość, klucze unikalne	skojarzenie dwóch rzeczy (login→punkty)

```
List<String> logins = new ArrayList<>();
logins.add("ania"); logins.add("ania"); // duplikat OK, size()==2

Set<String> unique = new HashSet<>();
unique.add("ania"); unique.add("ania"); // drugi NIE doda, size()==1
```

Czemu nie dwie osobne listy (loginy + wyniki) zamiast Map? Bo trzeba by ZAWSZE dodawać do obu list w tej samej kolejności — jeden błąd i indeksy się rozjeżdżają. Map łączy klucz+wartość w JEDEN, nierozłączny wpis.

ROZDZIAŁ POGŁĘBIONY

MAP — wszystko, co trzeba wiedzieć

Co to jest Map, dokładnie

Map<K, V> to struktura przechowująca PARY: każdy **klucz** (K) jest powiązany z jedną **wartością** (V). Klucze są UNIKALNE — nie może być dwóch wpisów z tym samym kluczem. Jeśli wstawisz drugi raz ten sam klucz, NADPISUJE poprzednią wartość, nie dodaje nowego wpisu.

Trzy główne implementacje

Implementacja	Porządek iteracji	Kiedy
HashMap	NIEPRZEWIDYWALNY (zależny od wewn. hashowania)	domyślny wybór, gdy porządek nieważny
LinkedHashMap	kolejność WSTAWIANIA	gdy chcesz zachować porządek (np. po sortowaniu)
TreeMap	sortowany po kluczu, automatycznie	gdy chcesz zawsze mieć klucze w porządku

Podstawowe operacje

```
Map<String, Integer> scores = new HashMap<>();

scores.put("ania", 10); // wstaw / NADPISZ jeśli klucz już istnieje
int s = scores.get("ania"); // odczyt wartości po kluczu – NPE jeśli klucza nie ma!
int s2 = scores.getOrDefault("x", 0); // bezpieczny odczyt, zwraca 0 jeśli klucza brak
boolean has = scores.containsKey("ania"); // czy klucz istnieje?
scores.remove("ania"); // usuń wpis po kluczu
int size = scores.size(); // liczba WPISÓW (par), nie kluczy/wartości osobno

// merge – bardzo przydatne do "dodaj punkt, albo wstaw 1 jeśli nowy"
scores.merge("ania", 1, Integer::sum); // jeśli "ania" istnieje: suma starej+1. Jeśli nie: wstaw 1
```

Pułapka NullPointerException: scores.get("nieistniejący") zwraca null dla obiektów (jak Integer), nie 0! Jeśli potem zrobisz int x = scores.get("x") bez sprawdzenia — NPE przy "rozpakowywaniu" null do int. Używaj getOrDefault().

Iterowanie po Map — TRZY sposoby

Map NIE JEST bezpośrednio iterowalna jak List (nie ma for (x : map) bez dodatkowego kroku). Trzeba wybrać, CO chcesz przeglądać:

```
// Sposób 1: tylko klucze
for (String key : scores.keySet()) {
    System.out.println(key + " -> " + scores.get(key)); // get() w pętli – działa, ale mniej efektywne
}
```

```
// Sposób 2: tylko wartości (gdy nie potrzebujesz kluczy)
for (Integer value : scores.values()) {
    System.out.println(value);
}

// Sposób 3 (NAJLEPSZY): klucz+wartość naraz, przez entrySet()
for (Map.Entry<String, Integer> entry : scores.entrySet()) {
    String key = entry.getKey();
    Integer value = entry.getValue();
    System.out.println(key + " -> " + value);
}
```

Co to jest Map.Entry? To "para" — jeden obiekt reprezentujący JEDEN wpis mapy (klucz + wartość razem). `entrySet()` zamienia całą Map na zbiór takich par — przydatne, gdy chcesz przejść po WSZYSTKICH parach naraz, bez osobnego `get()` dla każdego klucza.

Sortowanie Map po wartości — KROK PO KROKU

Map (zwłaszcza HashMap) NIE MA naturalnego porządku. Żeby posortować po wartości i ZACHOWAĆ wynik:

```
Map<String, Integer> sorted = leaderboard.entrySet().stream()
    // KROK 1: entrySet() – zamień Map na zbiór par (Stream nie umie przetwarzać Map bezpośrednio)
    .sorted(Map.Entry.<String,Integer>comparingByValue().reversed())
    // KROK 2: sortuj wg WARTOŚCI każdej pary, malejąco (.reversed())
    .collect(Collectors.toMap(
        Map.Entry::getKey,           // KROK 3a: jak wyciągnąć klucz z każdego Entry
        Map.Entry::getValue,        // KROK 3b: jak wyciągnąć wartość
        (a, b) -> a,                // KROK 3c: co przy duplikacie klucza (tu nigdy nie wystąpi)
        LinkedHashMap::new));       // KROK 3d: ⚠ KLUCZOWE – zwykła HashMap "zgubiłaby" porządek sortowania!
```

Dlaczego LinkedHashMap na końcu jest KONIECZNE: jeśli zamiast `LinkedHashMap::new` dasz domyślny `HashMap::new`, CAŁE sortowanie z kroku 2 zostanie "zmarnowane" — HashMap reorganizuje wpisy wewnętrznie, niezależnie od porządku wstawiania. LinkedHashMap PAMIĘTA kolejność wstawienia (stąd "Linked" w nazwie).

Po czym sortować — wzorce

```
// po wartości, rosnąco (domyślnie)
Map.Entry.<String,Integer>comparingByValue()

// po wartości, malejąco
Map.Entry.<String,Integer>comparingByValue().reversed()

// po kluczu (np. alfabetycznie)
Map.Entry.<String,Integer>comparingByKey()
```

ROZDZIAŁ POGŁĘBIONY

ITERATOR — wszystko, co trzeba wiedzieć

Co to właściwie jest Iterator

Iterator to obiekt, który "wie, gdzie aktualnie jest" podczas przechodzenia po kolekcji — jak zakładka w książce. Zamiast pytać kolekcję "daj mi element nr 5", pytasz Iterator "daj mi NASTĘPNY element" — Iterator sam pamięta, który był ostatni.

Najważniejsze: kiedy piszesz zwykłe `for (Player p : players)` (for-each), Java W TLE i tak tworzy Iterator i go używa — for-each to tylko "wygodniejszy zapis" dla Iteratora. Pisanie Iteratora "ręcznie" jest potrzebne tylko w konkretnych sytuacjach (patrz niżej).

Dwie metody, na których wszystko stoi

```
Iterator<Player> it = players.iterator(); // weź Iterator z kolekcji

while (it.hasNext()) {                    // hasNext() – "czy jest jeszcze COŚ do przejścia?" (true/false)
    Player p = it.next();                 // next() – "daj mi NASTĘPNY element" I PRZESUŃ zakładkę dalej
    System.out.println(p);
}
```

Kolejność ma znaczenie: `hasNext()` NIE przesuwa "zakładki", tylko SPRAWDZA czy coś jest dalej. `next()` faktycznie ZWRACA

element I przesuwa zakładkę o jeden dalej. Musisz wołać `hasNext()` PRZED każdym `next()`, żeby nie "spadł" z końca kolekcji (co rzuciłoby `NoSuchElementException`).

GŁÓWNY powód używania Iteratora: bezpieczne USUWANIE podczas przechodzenia

To jest sytuacja, gdzie zwykły `for-each` ZAWODZI:

```
for (ClientHandler c : clients) {
    if (!c.isConnected()) {
        clients.remove(c);    // ✗ RZUCI ConcurrentModificationException!
    }
}
```

Czemu to wybucha: `for-each` w tle używa Iteratora. Gdy MODYFIKUJESZ listę (przez `remove()` NA SAMEJ LIŚCIE) w trakcie, gdy Iterator próbuje iść dalej, "zauważa" że lista zmieniła się inaczej niż przez niego samego — i rzuca wyjątek jako ochronę przed niespójnym stanem.

Rozwiązanie — usuwaj PRZEZ Iterator, jego własną metodą `remove()`:

```
Iterator<ClientHandler> it = clients.iterator();
while (it.hasNext()) {
    ClientHandler c = it.next();
    if (!c.isConnected()) {
        it.remove();    // ✓ bezpieczne – Iterator WIE, że właśnie usuwa, aktualizuje swój stan
    }
}
```

Pułapka: `it.remove()` MUSI być wywołane ZARAZ PO `it.next()` w tej samej iteracji — usuwasz dokładnie TEN element, który właśnie dostałeś. Nie można wywołać `remove()` bez wcześniejszego `next()`. I NIE MIESZAJ — jeśli używasz `it` do iteracji, usuwaj TYLKO przez `it.remove()`, nigdy przez `clients.remove(c)` w tej samej pętli — to dałoby TEN SAM błąd, bo lista bazowa nadal "nie wie" o iteratorze.

Tabela decyzyjna: `for-each` vs Iterator

Sytuacja	Co użyć
Tylko PRZEGLĄDAM elementy, nic nie usuwam	<code>for-each</code> — prościej, krócej
Chcę USUNĄĆ niektóre elementy W TRAKCIE przechodzenia	Iterator + <code>it.remove()</code>
Chcę usunąć WSZYSTKIE spełniające warunek, bez pętli	<code>list.removeIf(p -> warunek)</code> — jeszcze prościej!

`removeIf` — najprostsza alternatywa, gdy nie potrzebujesz nic innego robić w pętli:

`clients.removeIf(c -> !c.isConnected());` — jedna linia, robi to samo co `Iterator+remove()`, gdy jedyne co chcesz to usunąć elementy spełniające warunek.

Iterator na Map — przez `entrySet()`

```
Iterator<Map.Entry<String, Integer>> it = scores.entrySet().iterator();
while (it.hasNext()) {
    Map.Entry<String, Integer> entry = it.next();
    if (entry.getValue() < 0) {
        it.remove();    // usuwa wpis Z MAPY, bezpiecznie
    }
}
```

Sortowanie kolekcji — `Comparable` vs `Comparator`

	<code>Comparable<T></code>	<code>Comparator<T></code>
Gdzie	WEWNĄTRZ klasy, którą sortujesz	Z ZEWNĄTRZ, niezależna klasa/lambda
Ile wersji	JEDNA, "naturalny" porządek	WIELE, różne porządki dla tej samej klasy

```
class Player implements Comparable<Player> {
    private int score;
    @Override
```



```
        public int compareTo(Player other) { return Integer.compare(this.score, other.score); }
    }
    Collections.sort(players); // użyje compareTo()
    players.sort(Comparator.comparing(Player::getLogin)); // porządek "na chwilę", po loginie
    players.sort(Comparator.comparing(Player::getScore).reversed()); // malejąco
    players.sort(Comparator.comparing(Player::getScore).thenComparing(Player::getLogin)); // remisy po loginie
```

3 SWITCH — pełny rozdział

ROZDZIAŁ POGŁĘBIONY

Czemu switch w ogóle istnieje

`switch` to "skrót" dla wielu `if-else if-else` sprawdzających TĘ SAMĄ zmienną na różne wartości. Zamiast pisać:

```
if (gesture == ROCK) { ... }
else if (gesture == PAPER) { ... }
else if (gesture == SCISSORS) { ... }
else { ... }
```

...piszesz krócej i czytelniej przez `switch`. Działa na: `int`, `String`, `enum`, i kilku innych typach.

Switch KLASYCZNY (starsza składnia, z `break`)

```
switch (gesture) {
    case ROCK:
        System.out.println("kamień");
        break; // ⚠ KONIECZNE – bez tego "przeleci" do następnego case!
    case PAPER:
        System.out.println("papier");
        break;
    default:
        System.out.println("nożyce");
}
```

Pułapka fall-through: jeśli ZAPOMNISZ `break`, wykonanie "przełączy" do NASTĘPNEGO case, nawet jeśli warunek tego case nie jest sprawdzany ponownie! Np. bez `break` w `ROCK`, program wypisałby "kamień" i "papier" razem, mimo że `gesture==ROCK`.

```
switch (x) {
    case 1:
        System.out.println("jeden");
        // BRAK break – wykonanie PRZEŁATUJE dalej!
    case 2:
        System.out.println("dwa");
        break;
}
// jeśli x==1: wypisze "jeden" ORAZ "dwa" (fall-through!)
// jeśli x==2: wypisze tylko "dwa"
```

Można grupować case'y, jeśli mają identyczną obsługę (świadome, kontrolowane wykorzystanie fall-through):

```
switch (day) {
    case SATURDAY:
    case SUNDAY:
        System.out.println("Weekend");
        break;
    default:
        System.out.println("Dzień roboczy");
}
```

Switch NOWOCZESNY — strzałka (Java 14+)

Nowa składnia z `->` — KAŻDY case wykonuje TYLKO to, co jest po strzałce, BEZ fall-through, BEZ potrzeby `break`:

```
switch (gesture) {
    case ROCK -> System.out.println("kamień");
    case PAPER -> System.out.println("papier");
    case SCISSORS -> System.out.println("nożyce");
}
```

Wiele wartości w jednym case, oddzielone przecinkiem (zamiast grupowania case po case):

```
switch (day) {
    case SATURDAY, SUNDAY -> System.out.println("Weekend");
}
```

```
default -> System.out.println("Dzień roboczy");
}
```

Gdy potrzebujesz WIĘCEJ niż jednej instrukcji w danym case — używasz bloku `{ }`:

```
switch (gesture) {
    case ROCK -> {
        System.out.println("kamień");
        licznikKamieni++;
    }
    case PAPER -> System.out.println("papier");
    default -> {} // pusty blok – "nic nie rób", ale syntaktycznie wymagany jeśli inne case mają {}
}
```

Switch jako WYRAŻENIE — zwraca wartość!

To jest inny tryb użycia — switch nie tylko WYKONUJE kod, ale może SAM BYĆ wartością, przypisaną do zmiennej przez `return` (w metodzie) lub bezpośrednio:

```
static Gesture fromString(String s) {
    return switch (s) { // "return switch" – całe wyrażenie switch JEST wartością do zwrócenia
        case "r" -> ROCK;
        case "p" -> PAPER;
        case "s" -> SCISSORS;
        default -> throw new IllegalArgumentException("Nieznany gest: " + s);
    }; // ŚREDNIK na końcu całego wyrażenia switch!
}
```

Albo przypisanie do zmiennej (bez return, jeśli nie jesteśmy w metodzie):

```
String opis = switch (gesture) {
    case ROCK -> "kamień";
    case PAPER -> "papier";
    case SCISSORS -> "nożyce";
};
```

Kiedy switch jako wyrażenie, kiedy jako instrukcja:

Wyrażenie (zwraca wartość, ma `;` na końcu) — gdy każdy case "produkuje" wartość, którą chcesz przypisać/zwrócić.

Instrukcja (klasyczna, wykonuje kod, NIE ma `;` po całości) — gdy każdy case "robi coś" (wypisuje, modyfikuje pole), niezależnie czy coś zwraca.

yield — gdy w wyrażeniu potrzebujesz bloku z logiką

Jeśli case w switch-wyrażeniu potrzebuje WIĘCEJ niż jednej linii, ZANIM "zwróci" wartość — używasz `yield` (nie `return`!):

```
String opis = switch (gesture) {
    case ROCK -> {
        System.out.println("licząc..."); // dodatkowa logika
        yield "kamień"; // "yield" = "to jest WYNIK tego case"
    }
    case PAPER -> "papier";
    default -> "nożyce";
};
```

Tabela decyzyjna — który switch wybrać

Sytuacja	Wybierz
Każdy case ma 1 prostą instrukcję, nie potrzebujesz wartości zwrotnej	Nowoczesny ze strzałką <code>case X -> akcja();</code>
Chcesz, żeby switch "zwrócił" wartość do zmiennej/return	Switch jako wyrażenie, z <code>;</code> na końcu
Piszesz w starszym kodzie/projekcie, konwencja z break	Klasyczny z <code>break</code> (pamiętaj o break!)
Kilka wartości ma robić to samo	Grupowanie: <code>case A, B -> ...</code> (nowy) lub <code>case A: case B:</code> (stary)

Najczęstszy błąd na kolokwium: mieszanie składni — pisanie `break;` w switch ze strzałkami (niepotrzebne, błąd składni) albo zapominanie `break;` w klasycznym switch (fall-through, błąd logiczny, KOMPILUJE SIĘ, ale działa źle).

4 Wyjątki

Checked vs unchecked

	Checked	Unchecked
Dziedziczy po	Exception	RuntimeException
Kompilator wymusza obsługę?	TAK — try-catch albo throws	NIE — kod skompiluje się bez nic
Przykład	IOException , FileNotFoundException	NullPointerException , ArithmeticException
Typowo oznacza	sytuację zewnętrzną, poza kontrolą kodu (plik, sieć)	błąd w logice programu (Twoja "wina")

Błąd składni vs wyjątek: to są RÓŻNE rzeczy! Błąd składni (np. brak średnika) = kod się NIE skompiluje, program nigdy nie wystartuje. Wyjątek = kod jest poprawny gramatycznie, kompiluje się, ale W TRAKCIE DZIAŁANIA coś idzie nie tak.

try-catch-finally — mechanika

```
try {
    int result = 10 / parseInput(input);    // (A) tu może wybuchnąć
    System.out.println(result);              // (B) NIE wykona się, jeśli (A) rzuci wyjątek
} catch (ArithmeticException e) {
    System.out.println("Błąd: " + e.getMessage());    // (C) tu program "skacze"
} finally {
    System.out.println("ZAWSZE się wykona");          // (D) niezależnie od (A)/(C)
}
System.out.println("Program działa dalej!");          // (E) program NIE crashuje, jeśli wyjątek złapany
```

Kluczowe: wyjątek złapany przez catch NIE zatrzymuje całego programu — przerywa TYLKO resztę bloku try, program kontynuuje PO całym try-catch.

finally wykonuje się ZAWSZE — czy try zakończył się normalnie, czy przez wyjątek (złapany lub nie). To jedyne miejsce gwarantujące wykonanie w OBU scenariuszach, bez duplikowania kodu w catch i po pętli w try.

try-with-resources

```
try (BufferedReader reader = new BufferedReader(new FileReader("plik.txt"))) {
    // ...
} catch (IOException e) {
    System.err.println("Błąd: " + e.getMessage());
}
// reader.close() wywoła się AUTOMATYCZNIE – przed catch, niezależnie czy był wyjątek
```

Warunek: obiekt musi implementować Closeable / AutoCloseable . Działa dla plików, socketów, połączeń z bazą danych.

Własne wyjątki

```
class DivisionByZeroException extends RuntimeException {    // unchecked – błąd WYWOŁUJĄCEGO
    public DivisionByZeroException(String message) { super(message); }
}

public int divideOrThrow(int a, int b) {
    if (b == 0) { throw new DivisionByZeroException("Nie można dzielić przez zero"); }
    return a / b;
}
```

throw = instrukcja rzucająca KONKRETNY wyjątek w danym miejscu. **throws** = deklaracja w sygnaturze metody, mówiąca "ta metoda MOŻE rzucić ten typ, Ty (wywołujący) musisz się tym zająć".

Wybór checked vs unchecked dla własnego wyjątku: jeśli problem da się uniknąć po stronie wywołującego (sprawdzić argumenty wcześniej) → unchecked. Jeśli zależy od czynników zewnętrznych (plik może zniknąć w trakcie) → checked.

5 JUnit

Struktura testu

```
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.BeforeEach;
import static org.junit.jupiter.api.Assertions.*;

class DuelTest {
    private Player p1, p2;
    private Duel duel;

    @BeforeEach // wykonuje się PRZED KAŻDYM testem, świeże dane
    void setUp() {
        p1 = new Player();
        p2 = new Player();
        duel = new Duel(p1, p2);
    }

    @Test
    void evaluateReturnsNullOnDraw() { // ARRANGE w setUp(), ACT poniżej
        duel.handleGesture(p1, Gesture.ROCK);
        duel.handleGesture(p2, Gesture.ROCK);
        assertNull(duel.evaluate()); // ASSERT
    }
}
```

Schemat AAA: **Arrange** (przygotuj dane) → **Act** (wykonaj to co testujesz) → **Assert** (sprawdź wynik).

Asercje

Asercja	Kiedy
<code>assertEquals(expected, actual)</code>	liczby/obiekty/String — UWAGA: expected PIERWSZE
<code>assertTrue(warunek) / assertFalse</code>	boolean (np. <code>isDuelling()</code>)
<code>assertNull(obj) / assertNotNull</code>	czy referencja jest/nie jest null
<code>assertThrows(Wyjątek.class, () -> {...})</code>	czy kod RZUCA dany wyjątek

Złota zasada: 1 test = 1 scenariusz. Remis i wygrana to DWA osobne testy, nie jeden z wieloma sprawdzeniami — żeby wiedzieć precyzyjnie, co nie działa, gdy test padnie.

6 Programowanie funkcyjne

Lambdy — składnia

Argumenty	Składnia	Przykład
0	<code>() -> ciało</code>	<code>() -> System.out.println("hej")</code>
1	<code>x -> ciało</code> (nawiasy opcjonalne)	<code>s -> s.isEmpty()</code>
2+	<code>(a, b) -> ciało</code> (nawiasy wymagane)	<code>(a, b) -> a + b</code>

W nawiasach jest NAZWA zmiennej, nigdy typ. Wiele instrukcji w ciele `→ { } i return`.

Interfejsy funkcyjne — wbudowane

Interfejs	Przyjmuje	Zwraca	Metoda
<code>Supplier<T></code>	nic	T	<code>.get()</code>
<code>Consumer<T></code>	T	nic	<code>.accept(x)</code>
<code>Function<T,R></code>	T	R	<code>.apply(x)</code>
<code>Predicate<T></code>	T	boolean	<code>.test(x)</code>
<code>Runnable</code>	nic	nic	<code>.run()</code>

effectively final — pułapka: lambda może użyć zmiennej z otoczenia TYLKO jeśli ta zmienna się nie zmienia po przypisaniu.
`for (int i=1; i<=5; i++) { pool.submit(() -> print(i)); }` — **✗** błąd kompilacji, "i" się zmienia.
Naprawa: `final int taskNumber = i;` NOWA zmienna w KAŻDEJ iteracji, potem użyj jej w lambdzie.

Wzorzec callback — np. `Duel.onEnd`

```
@FunctionalInterface
interface DuelEndListener { void onEnd(); }

class Duel {
    private DuelEndListener onEnd;
    public void setOnEnd(DuelEndListener listener) { this.onEnd = listener; }

    private void checkBothGesturesSet() {
        if (gesture1 != null && gesture2 != null && onEnd != null) {
            onEnd.onEnd(); // wywołanie callbacku
        }
    }
}

// z zewnątrz: duel.setOnEnd(() -> { ... logika po zakończeniu ... });
```

Podział odpowiedzialności: klasa z polem callbacku (`Duel`) wie KIEDY wywołać, nie wie CO. Kod z zewnątrz (`Server`) decyduje CO, nie pilnuje KIEDY. Zamiast własnego interfejsu można użyć `Runnable` (identyczna sygnatura: brak argumentów, brak wartości).

Stream API

```
List<String> result = players.stream()
    .filter(p -> p.getScore() > 10) // Predicate – zostaw spełniające warunek
    .sorted(Comparator.comparing(Player::getScore).reversed())
    .map(Player::getLogin) // Function – przekształć każdy element
    .limit(5)
    .collect(Collectors.toList()); // zbierz wynik
```

Pułapka: strumień da się skonsumować TYLKO RAZ. Po `collect()/forEach()` ponowne użycie tego samego Stream rzuca wyjątek — wywołaj `.stream()` od nowa na kolekcji źródłowej.

7 Wątki i synchronizacja

Thread, Runnable, start vs run

```
class ClientHandler implements Runnable { // ZALECANE – może jeszcze extends czegoś innego
    @Override
    public void run() { /* kod wątku */ }
}
new Thread(new ClientHandler(...)).start();
```

PUŁAPKA: start() vs run()

`t.run()` — ŹLE, wykonuje się w BIEŻĄCYM wątku, sekwencyjnie, jak normalna metoda.
`t.start()` — DOBRZE, tworzy NOWY wątek systemowy.

ExecutorService

```
int cores = Runtime.getRuntime().availableProcessors();
ExecutorService pool = Executors.newFixedThreadPool(cores);
for (int i = 0; i < cores; i++) {
    final int taskId = i; // kopia do lambdy – effectively final
    pool.submit(() -> doWork(taskId));
}
pool.shutdown(); // nie przyjmuj NOWYCH zadań
pool.awaitTermination(1, TimeUnit.MINUTES); // CZEKAJ aż WSZYSTKIE się skończą (throws InterruptedException)
```

Pula 3 wątki + 5 zadań → 3 startują od razu, 4 i 5 CZEKAJĄ w kolejce, aż wątek się zwolni.

join() — czekanie na pojedynczy Thread:

```
// ✓ RÓWNOLEGLE: najpierw OBA start, potem OBA join
t1.start(); t2.start();
t1.join(); t2.join();

// ✗ SEKWENCYJNIE (błąd!): start+join na raz dla każdego
t1.start(); t1.join(); // czekasz na t1 ZANIM t2 nawet wystartuje!
t2.start(); t2.join();
```

Synchronizacja

Race condition: 2+ wątki modyfikują TĘ SAMĄ strukturę naraz → operacja (np. `ArrayList.add()`) nie jest atomowa, wątki "zazębiają się" → zgubione dane / `ConcurrentModificationException`.

```
public synchronized void addClient(ClientHandler h) { clients.add(h); } // 1 wątek na raz na TYM obiekcie

// PROSTSZA alternatywa – bezpieczne z definicji:
List<ClientHandler> clients = new CopyOnWriteArrayList<>(); // dużo czytasz, rzadko modyfikujesz
Map<String, Integer> scores = new ConcurrentHashMap<>();
```

== vs equals dla Stringów (pułapka niezwiązana z wątkami, ale częsta):

`if (input == "EXIT")` ✗ — porównuje OBIEKT, nie treść.
`if (input.equals("EXIT"))` ✓ — porównuje TREŚĆ.
`==` na obiektach jest OK gdy CELOWO sprawdzasz "ten sam obiekt" (np. `c == sender`).

8 Sockets — serwer i klient

ServerSocket, accept()

```
ServerSocket serverSocket = new ServerSocket(5000);
while (true) {
    Socket clientSocket = serverSocket.accept();    // BLOKUJE aż przyjdzie klient
    // obsłuż w NOWYM wątku, żeby accept() mógł wrócić po następnego
}
```

`accept()` jest BLOKUJĄCE — to jest powód, czemu każdy klient idzie do osobnego wątku.

Pełny wzorzec Server + ClientHandler

```
class Server {
    private final List<ClientHandler> clients = new CopyOnWriteArrayList<>();

    public void listen() throws IOException {
        try (ServerSocket serverSocket = new ServerSocket(5000)) {
            while (true) {
                Socket clientSocket = serverSocket.accept();
                ClientHandler handler = new ClientHandler(clientSocket, this);
                clients.add(handler);           // 1. NAJPIERW dodaj do listy
                new Thread(handler).start();    // 2. POTEŁ odpal wątek
            }
        }
    }

    public void removeClient(ClientHandler h) { clients.remove(h); }
    public void broadcast(String msg, ClientHandler sender) {
        for (ClientHandler c : clients) {
            if (c == sender) continue;    // pomiń nadawcę
            c.send(msg);
        }
    }

    public static void main(String[] args) throws IOException { new Server().listen(); }
}

class ClientHandler implements Runnable {
    private final Socket socket;
    private final Server server;
    private PrintWriter out;    // POŁE — żeby send() miało dostęp z zewnątrz

    public ClientHandler(Socket socket, Server server) { this.socket = socket; this.server = server; }

    @Override
    public void run() {
        try {
            BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            this.out = new PrintWriter(socket.getOutputStream(), true);
            String line;
            while ((line = in.readLine()) != null) {    // null gdy klient ZAMKNIŁ połączenie
                out.println("echo: " + line);
                server.broadcast(line, this);
            }
        } catch (IOException e) { // rozłączenie "brutalne" — normalne
        } finally {
            server.removeClient(this);    // ZAWSZE — normalne zakończenie LUB wyjątek
        }
    }

    public void send(String msg) { out.println(msg); }
}
```

Czemu finally, nie kod po pętli w try: jeśli wyjątek wyleci w pętli, kod PO pętli w try NIGDY się nie wykona. finally jest JEDYNYM miejscem gwarantującym wykonanie w OBU scenariuszach.

Klient TCP

```

Socket socket = new Socket("localhost", 5000); // AKTYWNIŁEM łączy się
PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));

// jeśli klient I wysyła I odbiera – potrzeba 2 WĄTKÓW (1 wątek nie może czekać na 2 rzeczy naraz)
Thread receive = new Thread(() -> {
    try {
        String line;
        while ((line = in.readLine()) != null) { System.out.println(line); }
    } catch (IOException e) { System.out.println("blad"); }
});
receive.start();

Scanner scanner = new Scanner(System.in); // klawiatura, jak cin w C++
while (true) {
    String input = scanner.nextLine();
    if (input.equals("EXIT")) { break; }
    out.println(input);
}

```

Pliki binarne przez socket

```

// WYSYŁANIE – prefiks długości, potem dane
byte[] data = Files.readAllBytes(path);
DataOutputStream dos = new DataOutputStream(socket.getOutputStream());
dos.writeInt(data.length);
dos.write(data);

// ODBIERANIE
DataInputStream dis = new DataInputStream(socket.getInputStream());
int len = dis.readInt();
byte[] buffer = new byte[len];
dis.readFully(buffer); // czeka aż dotrze KOMPLETNY plik (read() samo może dać mniej bajtów!)

```

9 JavaFX

Stage, Scene, layouty

```
public class Window extends Application {
    @Override
    public void start(Stage stage) {
        VBox root = new VBox(10); // V = Vertical, jeden POD drugim
        root.getChildren().addAll(new Label("Hej"));
        stage.setScene(new Scene(root, 400, 300));
        stage.show(); // BEZ TEGO okno się NIE pokaże
    }
    public static void main(String[] args) { launch(args); }
```

Stage = okno (rama). **Scene** = zawartość w danym momencie — jeden Stage może zmieniać Scene, nie trzeba nowego okna. **VBox** (pionowo) / **HBox** (poziomo) — różnica widoczna tylko przy 2+ elementach.

Częsty błąd: `java.awt.Label` zamiast `javafx.scene.control.Label` — sprawdź IMPORTY, IDE czasem podstawia złą klasę o tej samej nazwie.

Kontrolki i listenery

```
Slider slider = new Slider(1, 15, 3); // (min, max, wartość startowa)
Label label = new Label("3");

slider.valueProperty().addListener((obs, oldVal, newVal) -> { // JavaFX SAM wywoła to przy zmianie (jak callback z r.6)
    int raw = newVal.intValue();
    label.setText(String.valueOf(raw));
});
```

Reguła ogólna: `xProperty().addListener((obs, old, new) -> {...})` działa dla każdej kontrolki (`valueProperty`, `textProperty`, itd.)

Canvas

```
Canvas canvas = new Canvas(400, 400);
GraphicsContext gc = canvas.getGraphicsContext2D();
gc.setFill(Color.WHITE); gc.fillRect(0,0,400,400); // fill = wypełnienie
gc.setStroke(Color.RED); gc.strokeLine(50,50,350,350); // stroke = kontur/linia
gc.fillOval(x-5, y-5, 10, 10); // koło wyśrodkowane na (x,y)
```

KLUCZOWA ZASADA: Canvas to TYLKO bitmapa, nic nie pamięta. Każde rysowanie DORYSOWUJE się na to co jest. Zawsze: 1) pamiętaj listę elementów w polu klasy, 2) czyść Canvas przed przerysowaniem, 3) przerysuj WSZYSTKO od zera w jednej metodzie.

```
private final List<double[]> dots = new ArrayList<>();
private double offsetX = 0, offsetY = 0;

// ZAPIS — odejmij offset (zapisz pozycję "czystą")
canvas.setOnMouseClicked(event -> {
    dots.add(new double[]{event.getX() - offsetX, event.getY() - offsetY});
    redraw();
});

// RYSOWANIE — dodaj offset
private void redraw() {
    gc.setFill(Color.WHITE); gc.fillRect(0,0,400,400); // wyczyść
    gc.setFill(Color.BLUE);
    for (double[] dot : dots) {
        gc.fillOval(dot[0]+offsetX-5, dot[1]+offsetY-5, 10, 10);
    }
}
```

Pułapka offsetu: bez odejmowania przy zapisie, offset "wliczy się dwa razy" przy kolejnym przesunięciu — elementy skoczą dalej niż

powinny.

Klawiatura i mysz

```
scene.setOnKeyPressed(event -> {      // NA SCENE, nie na Canvas! (koncept "focus")
    switch (event.getCode()) {
        case UP      -> offsetY -= 10;
        case DOWN    -> offsetY += 10;
        case LEFT     -> offsetX -= 10;
        case RIGHT    -> offsetX += 10;
        default -> {}
    }
    redraw();
});
canvas.requestFocus();    // dobra praktyka po stage.show()
```

Mysz na konkretnym Node (np. Canvas) — `event.getX()/getY()` względem TEGO elementu. Klawiatura na Scene, bo łapie klawisze niezależnie od tego, co ma fokus.

Checklist ratunkowy — 60 sekund przed paniką

- ☐ Błąd "no suitable method" na Label/Button → sprawdź import, czy to javafx, nie java.awt
- ☐ JavaFX nie odpala się przez zwykły Run → użyj javafx:run z panelu Maven albo VM options z --module-path
- ☐ NullPointerException przy Map.get() → użyj getOrDefault(), zwykły get() zwraca null dla brakującego klucza
- ☐ ConcurrentModificationException w pętli → usuwasz podczas for-each, użyj Iterator.remove() albo removeIf()
- ☐ Switch nie działa jak chcesz → sprawdź break (klasyczny) albo czy mieszasz składnię arrow ze starą
- ☐ Lambda nie kompiluje się w pętli for → effectively final, skopiuj zmienną do nowej w danej iteracji
- ☐ Wątki działają "sekwencyjnie" mimo start() → sprawdź czy join() nie jest wywołane zaraz po każdym start(), powinno być: wszystkie start(), potem wszystkie join()
- ☐ String porównanie nie działa → == zamiast .equals(), prawie zawsze .equals() dla Stringów
- ☐ Serwer "wisi" i nie przyjmuje drugiego klienta → obsługa klienta nie jest w osobnym wątku
- ☐ Canvas "smuży się" przy rysowaniu → brak czyszczenia (fillRect całości) przed przerysowaniem
- ☐ Mapa po sortowaniu "gubi" porządek → użyto HashMap zamiast LinkedHashMap na końcu collect()
- ☐ Klient zawiesza się i nie odbiera wiadomości serwera → wysyłanie i odbieranie w jednym wątku, potrzeba dwóch